

拟机和真实的物理主机几乎没有区别。系统虚拟化技术的核心是虚拟机监控器，它一般运行在 CPU 的最高特权级，直接控制硬件，并为上层软件提供虚拟的硬件接口，让这些软件“以为”自己运行在真实的物理主机上。

系统虚拟化技术主要包含三个方面，即 CPU 虚拟化、内存虚拟化和 I/O 虚拟化。

- CPU 虚拟化：为虚拟机提供虚拟处理器（Virtual CPU，vCPU）的抽象并执行其指令的过程。虚拟机监控器直接运行在物理主机上，使用物理 ISA，并向上层虚拟机提供虚拟 ISA。虚拟 ISA 可以与物理主机上的 ISA 相同，也可以完全不同。如果两者相同，虚拟机中的大多数指令可以在物理主机上直接执行，只有少数敏感指令需要特殊对待，因此具有较好的性能。如果两者不同，虚拟机中的每一条指令都必须通过虚拟机监控器进行软件模拟，翻译成对应物理主机上的指令。
- 内存虚拟化：为虚拟机提供虚拟的物理地址空间。在虚拟化环境中，虚拟机监控器负责管理所有物理内存，但又要让客户操作系统“以为”自己依然能管理所有物理内存。为此，虚拟机监控器引入了一层新的地址空间——“客户物理地址”，以与真实的物理地址区分。虚拟机监控器还提供了一种翻译机制，将虚拟机中“假”的物理地址翻译成“真”的物理地址。
- I/O 虚拟化：为虚拟机提供虚拟的 I/O 设备支持。在虚拟化环境中，虚拟机监控器负责管理所有的 I/O 设备，客户操作系统所管理的仅仅是虚拟机监控器提供的虚拟设备，虚拟机监控器需要将对虚拟设备的访问映射成对物理设备的访问。

14.1.2 虚拟机监控器的类型

1974 年，Gerald J. Popek 和 Robert P. Goldberg 提出，高效的系统虚拟化需要满足以下三个条件^[3]：

- 条件 1：虚拟机监控器为虚拟机内的程序提供与该程序的目标执行硬件完全一样的硬件接口。这意味着，所有能在物理主机上运行的程序，都应该能够运行在虚拟机上。
- 条件 2：虚拟机运行时的性能只比无虚拟化的情况下略微差一点。
- 条件 3：虚拟机监控器控制所有物理系统资源，例如 CPU、内存、存储、网络等资源。换句话说，虚拟机不能使用任何不属于它的资源，虚拟机监控器也可以在任何时候回收资源。

Goldberg 将虚拟机监控器分为两种类型，分别是 Type-1 和 Type-2，如图 14.1 所示。Type-1 型虚拟机监控器直接运行在最高特权级，可直接控制物理资源，并负责实现调度和资源管理等功能。可以将 Type-1 虚拟机监控器理解为一种特殊的操作系统，它所管理的“进程”就是虚拟机。Type-1 虚拟机监控器的典型代表是 Xen[Ⓔ]。

Ⓔ 参见 <https://xenproject.org/>。

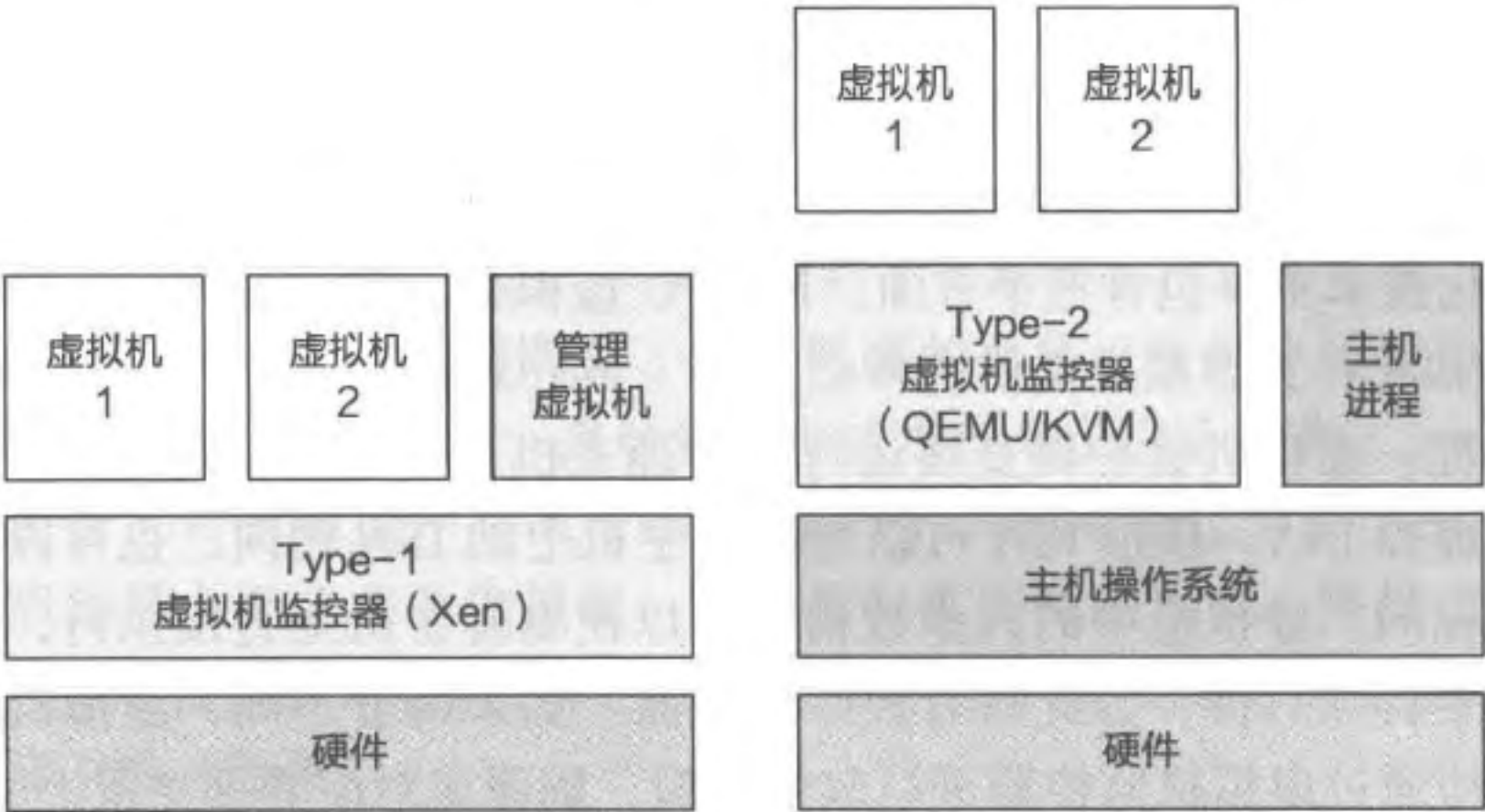


图 14.1 两种虚拟机监控器：Type-1 和 Type-2

Type-2 型虚拟机监控器需要依托一个宿主操作系统，比如 Linux 或 Windows。Type-2 型监控器可以复用宿主操作系统中的调度和资源管理等功能，从而专注于提供虚拟化相关的功能。与 Type-1 不同的是，Type-2 虚拟机监控器更像是宿主操作系统上的一个进程，典型代表是 QEMU。其中，操作系统内核（例如 Linux）负责资源管理，QEMU 负责提供核心的虚拟化功能。

14.2 “下陷 – 模拟” 方法

本节主要知识点

- 什么是“下陷 – 模拟”方法？
- 进程和虚拟机有什么相似点和不同点？
- 如何在多物理 CPU 的主机上运行多个虚拟 CPU？

如何把一台计算机虚拟化成多台计算机，使每台虚拟的计算机都能运行自己的操作系统？我们已经知道，操作系统和应用程序都是软件，由一条条指令组成。无论是进行运算、读写内存还是访问设备，都是由 CPU 通过执行这些指令来完成的。所以虚拟化一台计算机，最重要的是虚拟化 CPU 的接口，将一个物理 CPU 虚拟化成多个虚拟 CPU (vCPU)。我们也已经知道，CPU 的接口分为用户 ISA 和系统 ISA，操作系统通过进程这一抽象，支持多个应用在一个 CPU 上同时运行，即已经实现了用户 ISA 的虚拟化。如此可以推导出一个结论：如果在进程的基础上，再增加对系统 ISA 的虚拟化，就可以实现对整台计算机的虚拟化。换句话说，如果我们能够为进程增加系统 ISA 虚拟化的能力，将其扩展成一种新的“虚拟机进程”，然后将一个操作系统运行在这个虚拟机进程中，保

证这个操作系统执行的所有用户 ISA 指令和系统 ISA 指令会且仅会影响当前的虚拟机进程，而不会对其他进程或虚拟机进程产生影响，那么，我们就能够在同一台计算机中运行多个客户操作系统，也就是实现了系统虚拟化。

此时，客户操作系统运行在用户态的“虚拟机进程”中，却并不知道自己被降权，而是依然使用系统 ISA。CPU 一旦在用户态执行系统 ISA 的特权指令，便会触发 CPU 异常，下陷（Trap）到真正的内核来处理。内核会根据导致下陷的系统 ISA 指令的具体语义，用软件的方式模拟（Emulate）这条指令对相应“虚拟机进程”所产生的效果，然后返回用户态的下一条指令继续执行。这种实现系统虚拟化的方法称为下陷 - 模拟（Trap-and-Emulate）。

为了更深入地理解 CPU 虚拟化的设计和实现细节，本节以一个功能不断演进的虚拟机为例，分五个不同的版本对基于“下陷 - 模拟”方法的 CPU 虚拟化进行介绍。虚拟机的功能在这些版本中不断演进，每一版本都在上一版本的基础上添加了新的功能。最终版的虚拟机支持多个 vCPU，能运行虚拟机内核和多个用户态线程，虚拟机内核能实现用户态线程的调度，而用户态线程则可通过系统调用以及时钟中断的方式与虚拟机内核进行交互。

为了专注于理解 CPU 虚拟化，本节对虚拟机的功能进行了两点简化。首先，假设每个虚拟机只使用一个页表，且不会使用外部设备，因此虚拟机不处理外部设备的中断，只处理时钟中断。其次，本节讨论的是一个简化的硬件架构，表 14.1 展示了该简化架构的部分系统 ISA，本节还假设这些指令和寄存器在用户态执行 / 访问时均会造成下陷并进入内核态。

表 14.1 简化硬件架构的部分系统 ISA

类 型	指令名 / 寄存器名	描 述
指令	svc	系统调用
	eret	系统调用返回
寄存器	VBAR	异常向量表基地址寄存器
	IRQ_ON	中断开关寄存器，值为 1 表明打开中断
	ELR	异常返回目标地址寄存器

14.2.1 版本零：用进程模拟虚拟机内核态

在这一版本中，我们直接使用进程来模拟虚拟机。由于进程只有用户态，仅能运行用户 ISA，因此这个版本虚拟机的功能严重受限。首先，由于只支持一个特权级——内核态，因此虚拟机内没有内核态与用户态之间的特权切换。其次，虚拟机内核只用到用户 ISA 而不会用到系统 ISA，也就是说该内核运行过程中不会使用任何特权指令。图 14.2 展示了一个简化的例子：用户态只有一个虚拟机，虚拟机内只包含运行 while

循环的内核。由于虚拟机运行在用户态，因此此时虚拟机的内核态不是处理器硬件上的内核态，而是由虚拟机监控器为虚拟机模拟出的一种软件上的内核态。

可以看到，在这个版本中主要的变化是对术语名做了一次映射：进程改名为“虚拟机”，进程的用户态改名为“虚拟机的内核态”；操作系统改名为“虚拟机监控器”；从用户态下陷到内核态改名为“从虚拟机下陷到虚拟机监控器”，又称“VM Exit”；从内核态进入用户态改名为“从虚拟机监控器进入虚拟机”，又称“VM Entry”。此外，原本的 process 结构体（即 PCB）通过封装的方式改名为“虚拟机的上下文” VM_CTX。

运行于内核态的虚拟机监控器需要配置定时器，使其以固定的时钟周期打断虚拟机的执行，通过时钟中断下陷确保虚拟机监控器得以运行。在虚拟机的运行过程中，如果发生时钟中断并引起虚拟机下陷（VM Exit），处理器硬件将进行特权级切换并进入内核态，然后调用虚拟机监控器已经提前注册的时钟中断处理函数。中断处理函数首先保存虚拟机的上下文，将此时虚拟机使用的所有寄存器（通用寄存器、PC 寄存器、SP 寄存器等）的值存入 VM_CTX 的 process 中。中断处理函数处理完此中断下陷，重新从 VM_CTX 中将虚拟机寄存器的值加载进寄存器中，并恢复虚拟机的执行。

通过在进程外“包一层”的方式实现虚拟机，可以让虚拟机复用进程的上下文保存、恢复、调度等功能，同时可以让我们集中精力来处理运行真正的虚拟机内核所需要解决的问题。

14.2.2 版本一：模拟时钟中断

相对于版本零，版本一的虚拟机需要增加对虚拟时钟中断的支持，从而为在下一版本的虚拟机中运行用户态线程打下基础。为了处理中断，虚拟机监控器需要为虚拟机提供四种能力。第一，虚拟机能提供中断处理函数，并在异常向量表内记录该函数的地址。第二，虚拟机可以配置定时器，该定时器会周期性地触发时钟中断。第三，虚拟机能控制是否屏蔽中断。第四，如果中断发生时虚拟机未屏蔽中断，异常向量表内记录的中断处理函数会被执行。

由于 CPU 上只有一个物理定时器，并且已被虚拟机监控器使用，因此无法允许客户操作系统直接控制该定时器。为了支持在虚拟机内部使用定时器，一种方法是依然由虚

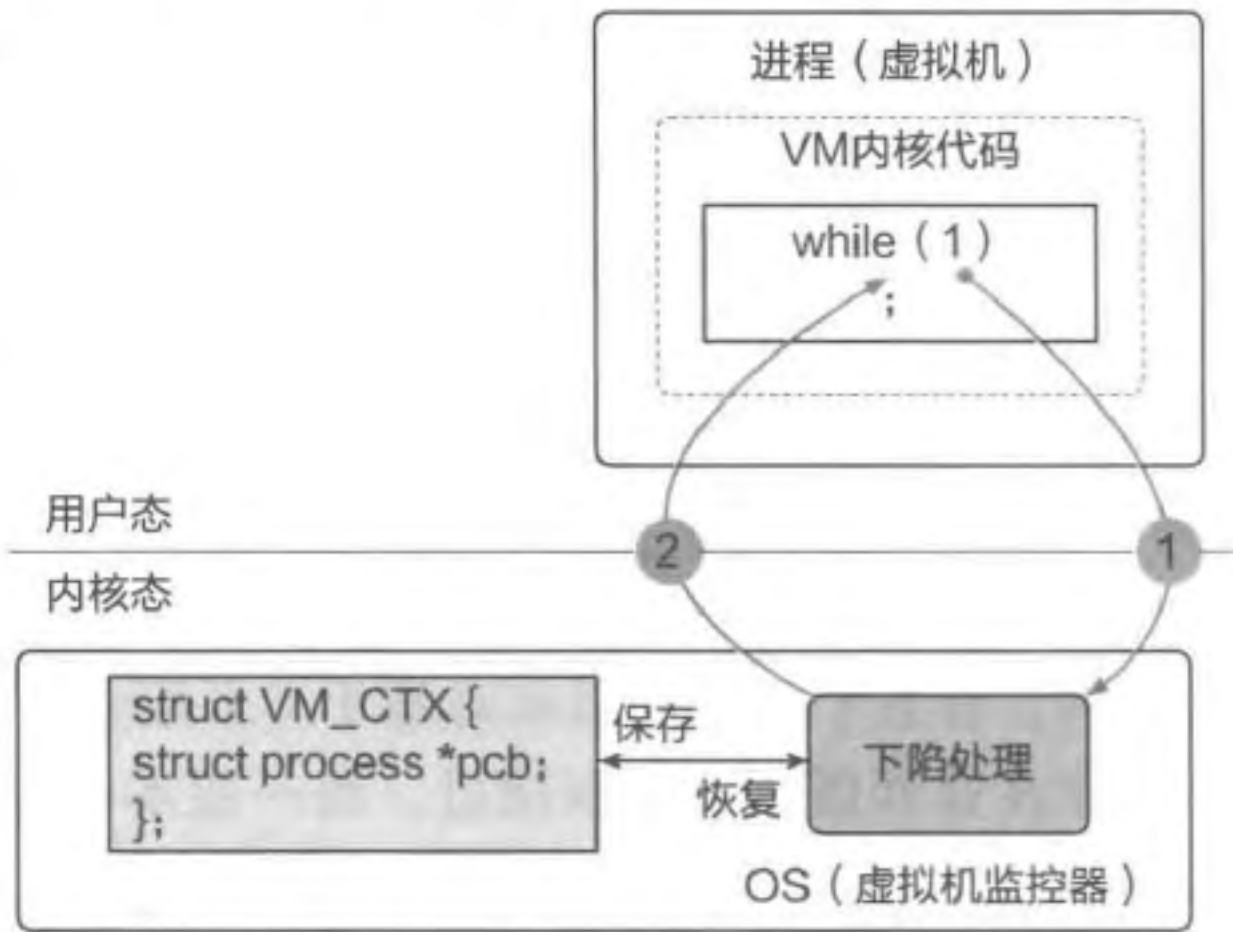


图 14.2 版本零：虚拟机只在内核态运行简单代码

拟机监控器控制定时器，并由它间接地为虚拟机提供“虚拟定时器”。即每当虚拟机内核通过系统 ISA 对定时器进行配置，就会触发下陷并进入内核态，此时虚拟机监控器针对具体的下陷原因进行相关模拟操作。虚拟机监控器也不允许虚拟机直接得到物理时钟中断的通知，而是通过模拟的方法为虚拟机提供虚拟时钟中断的抽象。

图 14.3 展示了一个具体的例子。假定虚拟机监控器已经将时钟配置为每隔一段时间（如 1 毫秒）触发一次中断。为了支持虚拟机的虚拟时钟中断，我们在虚拟机上下文 VM_CTX 中添加了两个新的变量，分别为 VBAR 和 IRQ_ON，用于模拟对应的 CPU 寄存器。虚拟机的初始化阶段包含 4 步：

- 第一步：虚拟机内核试图将包含时钟中断处理函数的异常向量表基地址写入异常向量基地址寄存器 VBAR。由于写入该寄存器的指令 `msr` 属于系统 ISA，会触发虚拟机下陷并进入虚拟机监控器。
- 第二步：虚拟机监控器获取写入的地址，将其存储在 VM_CTX 的 VBAR 中，返回虚拟机继续执行。
- 第三步：虚拟机内核决定打开中断，它向 IRQ_ON 系统寄存器中写入 1 时，会再次下陷，理由同第一步。
- 第四步：同样，虚拟机监控器并不会直接修改真正的 IRQ_ON 寄存器，而是在 VM_CTX 的 IRQ_ON 中记录虚拟机试图写入的值，并恢复虚拟机的执行。

完成初始化后，若客户虚拟机正常运行时发生物理时钟中断，处理步骤如下：

- 第五步：虚拟机执行过程中若触发物理时钟中断，此中断引起虚拟机下陷，最终调用虚拟机监控器的下陷处理函数。此函数检查 VM_CTX 内保存的 IRQ_ON 值，如果发现该值为 1，表明虚拟机未屏蔽中断。为了给虚拟机插入虚拟时钟中断，虚拟机监控器查询 VM_CTX 中的 VBAR，并找到虚拟机注册的时钟中断处理函数地址（即图 14.3 中的 `irq_handler`），最终将 ELR 寄存器设置为此函数地址^①。
- 第六步：虚拟机恢复执行并回到用户态，虚拟机此时立即执行 `irq_handler` 的代码对时钟中断进行处理。它首先需要保存下陷前内核的上下文，读取所有寄存器的值并压入栈中^②。保存完寄存器状态后，虚拟机开始处理时钟中断（如图 14.3 展示的统一时钟中断次数）。在退出中断处理函数前，虚拟机将栈中保存的值写入寄存器中（如把即将运行的指令地址写入 ELR）。当写入系统寄存器时，同样会多次触发下陷模拟过程，虚拟机监控器会将这些寄存器的值存入虚拟机的数据结构 VM_CTX 中。
- 第七步：虚拟机调用 `eret` 指令，触发下陷。

① 这段代码并未在图 14.3 中给出。

② 由于一些寄存器只能在内核态访问，读取它们的值时同样会触发下陷并进入内核态，虚拟机监控器在处理下陷时检查虚拟机试图读取的寄存器，将 VM_CTX 内保存的对应寄存器值返回虚拟机。这些下陷在图 14.3 中并未给出。

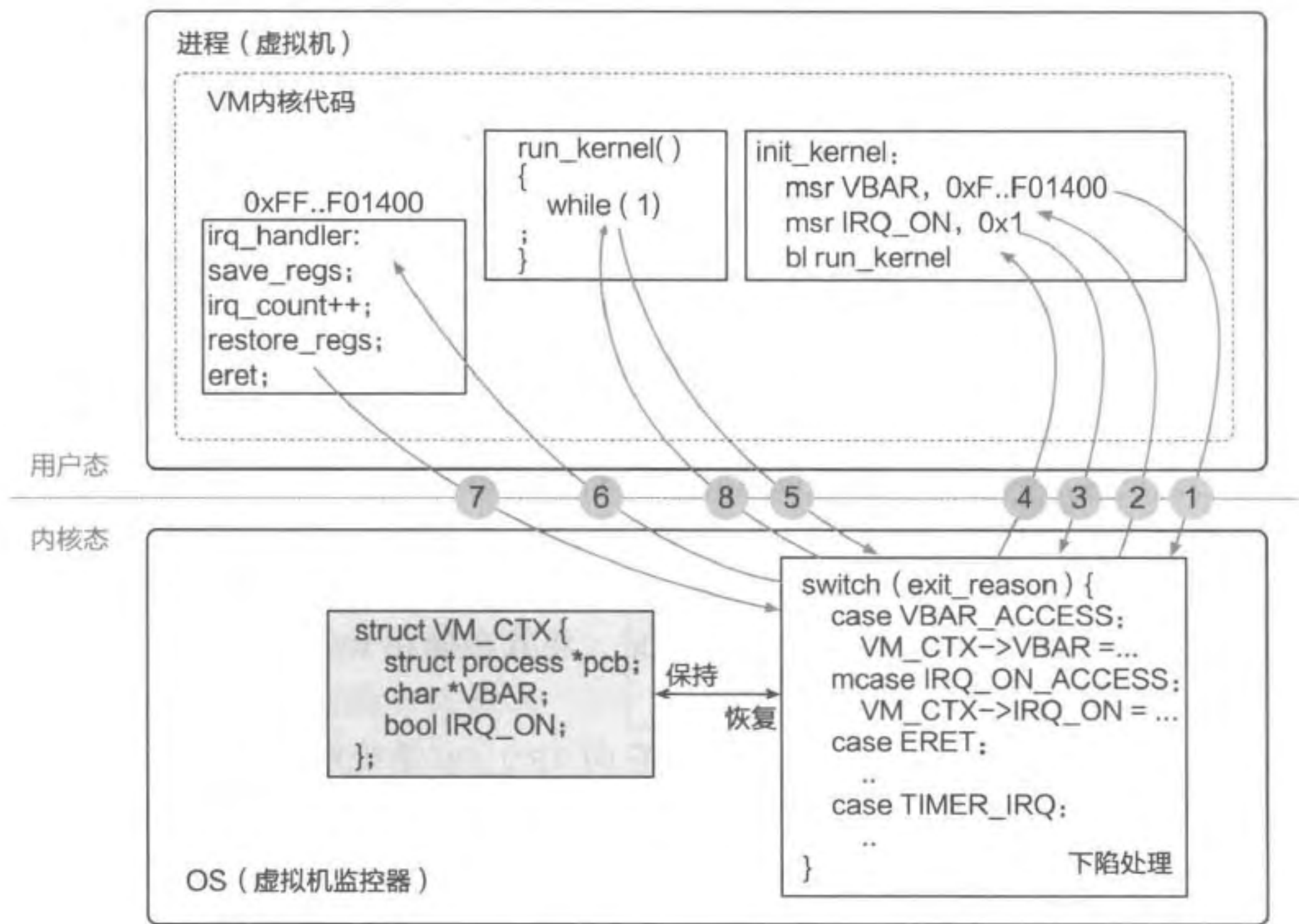


图 14.3 版本一：虚拟机内部支持时钟中断

- 第八步：虚拟机监控器将此前保存的 ELR 值写入物理 ELR 寄存器并回到用户态，虚拟机则会接着从第五步下陷时的地址继续往后执行。

请注意：在上述过程中，如果虚拟机内核在 `IRQ_ON` 寄存器中写入 0（表明屏蔽了中断），并不会真的屏蔽物理时钟中断。这是因为对虚拟机来说，该寄存器仅仅是内存中 `VM_CTX` 的一个字段，而不是真正的物理 `IRQ_ON` 寄存器。也就是说，客户操作系统只能影响自己所在虚拟机的状态。

14.2.3 版本二：模拟用户态与系统调用

接下来，我们在版本一的基础上为虚拟机添加单一用户态线程的支持，使虚拟机既可在（虚拟的）内核态运行操作系统内核，也可在（虚拟的）用户态运行应用的线程代码。当用户态线程运行时，它会执行用户 ISA，调用系统调用，也可能在运行时被时钟中断打断执行流并陷入虚拟机内核。

从运行的特权级视角来看，虚拟机的内核态与用户态本质上并无区别，二者均运行于物理的用户态，而虚拟机监控器通过“下陷 - 模拟”方法实现了虚拟的内核态和用户态。虚拟机监控器在 `VM_CTX` 中新增了 `curr_mode`（1 为内核态，0 为用户态），用于记录当前虚拟机内的虚拟特权态，并根据该变量的值决定如何处理虚拟机下陷。当在虚

拟内核态内因执行系统 ISA 而下陷时，如果虚拟机监控器根据 `curr_mode` 发现造成此下陷的是虚拟内核态，则会进行相应的功能模拟。如果虚拟机监控器发现造成下陷的是虚拟用户态，则不会进行功能模拟，而是将控制流转发至虚拟内核态，交由其进行处理。

如图 14.4 所示，`curr_mode` 的初始值为 1（即虚拟内核态），并首先运行虚拟机内核的 `run_kernel` 函数。该函数将时钟中断处理函数和系统调用处理函数的基地址写入 `VBAR` 寄存器。该寄存器的模拟过程与版本一中相同。在此版本中，虚拟机运行时的简要步骤如下：

- 第一步：虚拟机内核创建一个用户态线程，并初始化该线程的上下文 `thread_ctx`。为了执行此线程的代码，虚拟机内核需要将 `thread_ctx` 中的值加载至寄存器中，之后执行 `eret` 指令准备进入虚拟用户态。但是此指令在用户态执行会触发下陷，从而进入内核态。
- 第二步：虚拟机监控器在处理该下陷时首先保存虚拟机上下文，此时保存的是虚拟机内核为用户态线程设置的寄存器值。虚拟机监控器接着将 `VM_CTX` 的 `curr_mode` 改为 0，返回虚拟用户态执行用户态线程的代码。
- 第三步：执行用户态线程时，有两种方式可回到虚拟机内核态。第一种方式是用户态线程主动调用 `svc` 指令申请虚拟机内核的系统调用服务，第二种方式是被时钟中断打断执行流（如图 14.4 中执行 `while` 循环时被时钟中断），被动地陷入虚拟机内核。这两种方式都会触发下陷，进入虚拟机监控器的下陷处理函数。
- 第四步：虚拟机监控器根据下陷的原因调用对应的虚拟机内核处理函数。例如，如果下陷由 `svc` 造成，虚拟机监控器通过 `VM_CTX` 记录的 `VBAR` 查询到 `trap_handler` 地址，之后进入虚拟机内核执行该函数。若下陷是由于时钟中断，则进入 `irq_handler` 执行。
- 第五步：虚拟机内核在处理完系统调用或时钟中断后，会执行 `eret` 并下陷到虚拟机监控器。
- 第六步：虚拟机监控器最终回到虚拟机用户态，恢复用户态线程继续运行。

细心的读者可能已经发现，虚拟内核态与虚拟用户态都运行在用户态，那么如何保证隔离呢？一种方法是：当从虚拟内核态切换到虚拟用户态时，虚拟机监控器移除进程页表中虚拟机内核相关的映射；当发生虚拟机下陷时，再次恢复相关映射。这样，当处于虚拟用户态时，无法访问到任何属于虚拟内核态的内存，从而实现了对虚拟机内核的保护。关于内存虚拟化，会在后文进一步介绍。

14.2.4 版本三：虚拟机内支持多个用户态线程

为了支持多用户态线程的运行，虚拟机内核需要有支持多线程调度的能力。虚拟机内核首先创建一个用户态线程队列 `runq`，其中每个元素均为 `thread_ctx`。然后新增了一个调度函数——`schedule`，用于从队列中选择一个可执行的用户态线程。这里，

我们假设 `schedule` 仅实现了简单的时间片轮转调度策略。

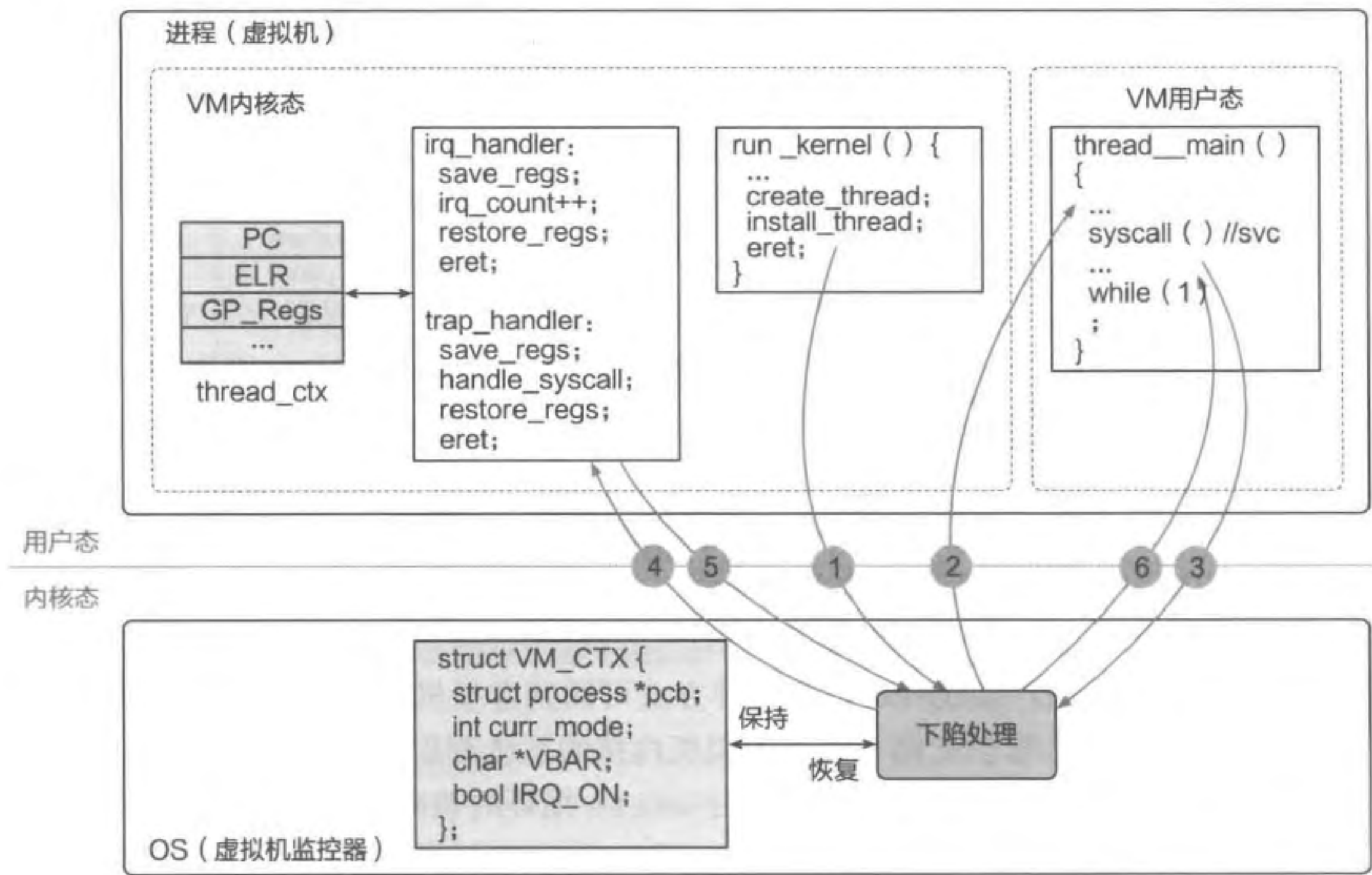


图 14.4 版本二：虚拟机内支持运行单一用户态线程。该用户态线程可主动调用 `svc` 或被动地通过时钟中断进入虚拟机内核

图 14.5 展示了虚拟机内多线程切换的过程。和之前一样，每当物理时钟中断打断虚拟机用户态线程时，会触发下陷并最终进入虚拟机内核的时钟中断处理函数 `irq_handler`。虚拟机内核调用 `schedule` 函数进行线程调度，选择下一个执行的用户态线程。在回到用户态前，虚拟机内核会将新的用户态线程 `thread_ctx` 中保存的上下文信息加载进寄存器中，并最终调用 `eret` 恢复虚拟用户态的执行。

从以上过程中可以看出，多用户态线程的功能全部在虚拟机内部实现，本版本中的虚拟机监控器相对于上一版本来说，能力并无变化。虚拟机监控器甚至对多用户态线程是无感知的，因为它只负责转发虚拟内核态与虚拟用户态之间的交互，无须得知虚拟用户态运行的具体线程信息。

14.2.5 版本四：用线程模拟多个 vCPU

现在，我们希望支持一个虚拟机使用多个 vCPU，从虚拟机的视角来看，每个 vCPU 上可“同时”运行不同的用户态线程。为了支持多 vCPU，虚拟机内核为每个 vCPU 维护一个不同的调度队列 `runq`。每个 vCPU 对应一个队列，其中保存会在此 vCPU 上运行

的所有用户态线程的上下文 thread。虚拟机内核在每个 vCPU 上独立运行，在调度时只访问本地的用户态线程调度队列。

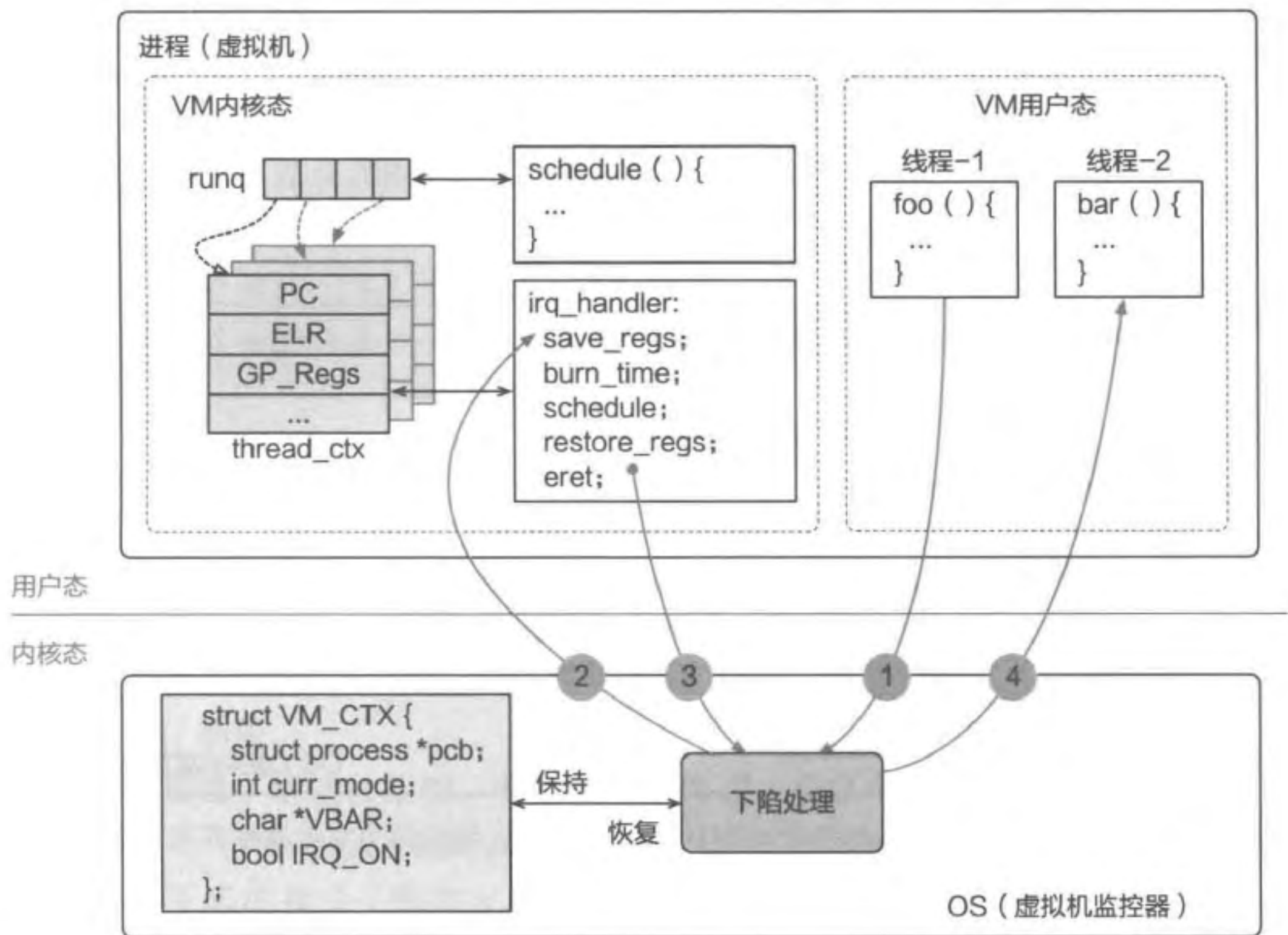


图 14.5 版本三：虚拟机内支持运行多个用户态线程并在线程间切换。图中展示了用户态线程切换前后触发下陷的相关控制流

每个 vCPU 为一个独立的执行实体，拥有独立的寄存器，所有 vCPU 共享虚拟机的地址空间，此概念与第 6 章介绍的线程类似。因此，为了支持多 vCPU，可以借助操作系统内已有的线程抽象。具体来说，从虚拟机的视角来看，它运行着多个 vCPU，每个 vCPU 上可“同时”运行不同的用户态线程。而从虚拟机监控器的视角来看，一个 vCPU 本质上只是一个线程，vCPU 的状态以及调度都借助线程实现。

如图 14.6 所示，我们对 VM_CTX 的定义进行升级，其中新增了 vCPU 结构体。该结构体大部分成员来自版本三的 VM_CTX，而 tcb 结构体为第 6 章中定义的线程，每个 vCPU 的寄存器状态可以保存在此结构体中。借助 tcb 结构体，虚拟机监控器可通过操作系统已有的线程调度机制间接地实现 vCPU 的调度。我们甚至可以进一步使用第 7 章介绍的多核调度机制，实现在多物理处理器（Physical CPU，pCPU）上运行不同的 vCPU。

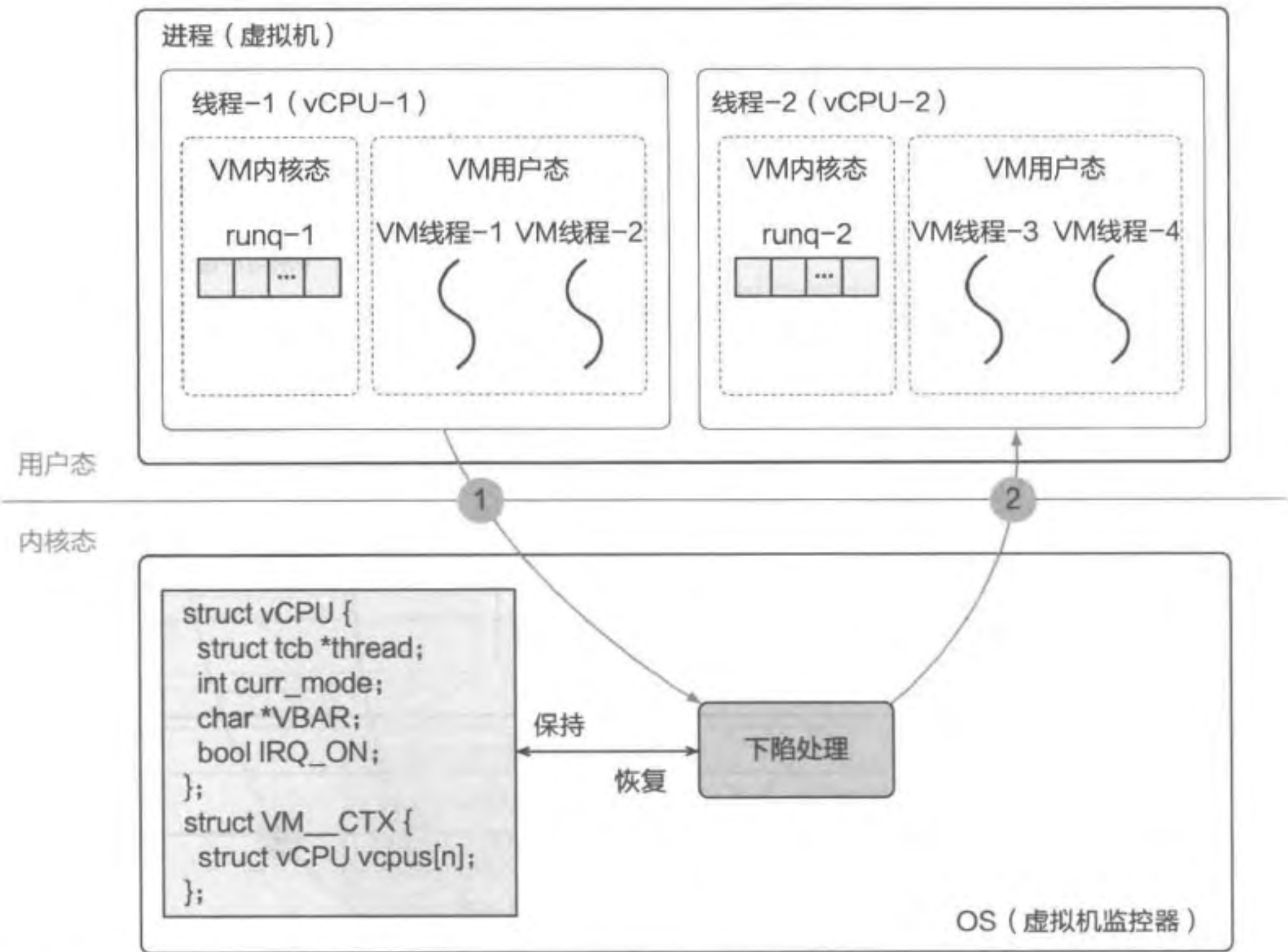


图 14.6 版本四：虚拟机支持运行多个 vCPU。图中的 tcb 结构体为第 6 章中定义的线程控制块

14.2.6 小结

如表 14.2 所示，我们在本节中通过迭代演进的方式介绍了一个功能不断丰富的虚拟机抽象，它逐渐具备了时钟中断、用户态线程、多线程切换、多 vCPU 支持等能力。借助多个 vCPU 的抽象，虚拟机监控器可通过调度的方法将多个 vCPU 同时运行在不同的 pCPU 上，从而充分利用多核机器的计算能力。为了支持这些能力，虚拟机监控器通过下陷 - 模拟的方法捕捉虚拟机的系统 ISA 下陷，并提供针对性的功能模拟。在进行功能模拟时，虚拟机监控器可复用已有操作系统的功能，例如进程和线程的调度、寄存器的上下文保存与恢复等。

表 14.2 五个版本的功能演进

版本名	用户态 ISA	时钟中断	用户态线程	多用户态线程	多 vCPU
版本零	✓				
版本一	✓	✓			
版本二	✓	✓	✓		
版本三	✓	✓	✓	✓	
版本四	✓	✓	✓	✓	✓

14.3 CPU 虚拟化

本节主要知识点

- 什么是可虚拟化架构？
- 如何通过解释执行的方法解决不可虚拟化架构的缺陷？
- 什么是动态二进制翻译？
- 什么是扫描和翻译？
- 什么是半虚拟化技术？它的优势是什么？
- 什么是硬件虚拟化技术？如何使用硬件虚拟化技术？

14.3.1 可虚拟化架构与不可虚拟化架构

在上一节中，我们假设在用户态执行的敏感指令都会造成下陷并进入特权态。然而在许多体系结构中，这一假设并不成立，也就是说敏感指令在用户态执行时无法触发下陷。因此，在这种架构中，我们无法直接实现下陷 - 模拟方法，也难以正确实现 14.2.5 节描述的 vCPU 抽象。

为了更详细地了解这种架构，我们首先需要进一步区分特权指令和敏感指令：

- 特权指令 (Privileged Instruction)：在用户态执行时会触发下陷的指令，包括主动触发下陷的指令（例如 `svc`）和不允许在用户态执行的指令（例如写入只读内存）等。
- 敏感指令 (Sensitive Instruction)：管理系统物理资源或者更改 CPU 状态的指令，通常包括以下类型：
 - 读写特殊寄存器或执行特殊指令更改 CPU 状态。例如在 x86 架构中修改 CR0 或 CR4 寄存器，在 ARM 架构中修改 SCTRL_EL1 寄存器；在 x86 架构中执行 HLT 指令，在 AArch64 架构中执行 wfi 指令等。
 - 读写敏感的内存，例如读写未映射的内存、写入只读页面等。
 - 执行 I/O 指令，例如 x86 架构中的 `in` 和 `out` 指令等。

可虚拟化架构的特征是所有敏感指令都是特权指令，即所有的敏感指令在非特权级执行时都会触发下陷。不满足这个定义的架构则称为不可虚拟化架构。20 世纪 70 和 80 年代的硬件设计师并未仔细考虑对虚拟化的支持，导致虚拟机监控器无法完全捕捉到客户操作系统的行为，也就不能提供相应的虚拟化功能。例如，在 AArch32 中，操作系统可使用 `cps` 指令修改 CPU 状态 PSTATE 中的 AIF 位来打开或关闭外部中断。该指令只有执行在特权级（如 EL1）才会生效，如果执行在 EL0 则不会产生任何作用，也不触发任何异常，而是会被硬件忽略（Silent Fail）。AArch32 和早期的 x86 都是不可虚拟化架

构，因此无法简单地通过下陷 - 模拟来实现系统虚拟化。

小知识：x86 中的 17 条不可虚拟化指令

x86 架构在最初设计和实现时，并没有充分考虑对虚拟化的支持。有 17 条敏感指令在用户态执行时不会触发任何下陷，也就意味着不会被虚拟机监控器捕捉^[4]。

下面我们将介绍 5 种弥补不可虚拟化架构缺陷的方法，分别是解释执行、动态二进制翻译、扫描 - 翻译、半虚拟化技术、硬件虚拟化技术。

14.3.2 解释执行

解释执行的方法是依次取出虚拟机内的每一条指令，然后用软件模拟这条指令的执行效果。该方法不依赖下陷，所有的指令都被虚拟机监控器模拟执行。值得注意的是，该方法模拟的是指令的效果，它并不一定需要精准地复现一条指令在硬件中的执行细节。

图 14.7 展示了指令模拟方法的简要过程。虚拟机内部的代码区域只是设置成可读权限（不可执行），由虚拟机监控器依次模拟每一条指令。

- 第一步：虚拟机监控器读取当前虚拟机的虚拟程序计数器（PC），该 PC 是虚拟机监控器模拟的虚拟寄存器。
- 第二步：根据 PC 的值找到待模拟的指令，并将指令解码（判断是何种类型的指令）。
- 第三步：指令模拟器根据解码的结果找到相关的指令模拟函数。
- 第四步：指令模拟函数读取虚拟机相关寄存器，运行指令模拟函数，并更新相关内存或虚拟寄存器中的值。
- 第五步：指令模拟器更新 PC，使其指向下一条指令，回到第一步。

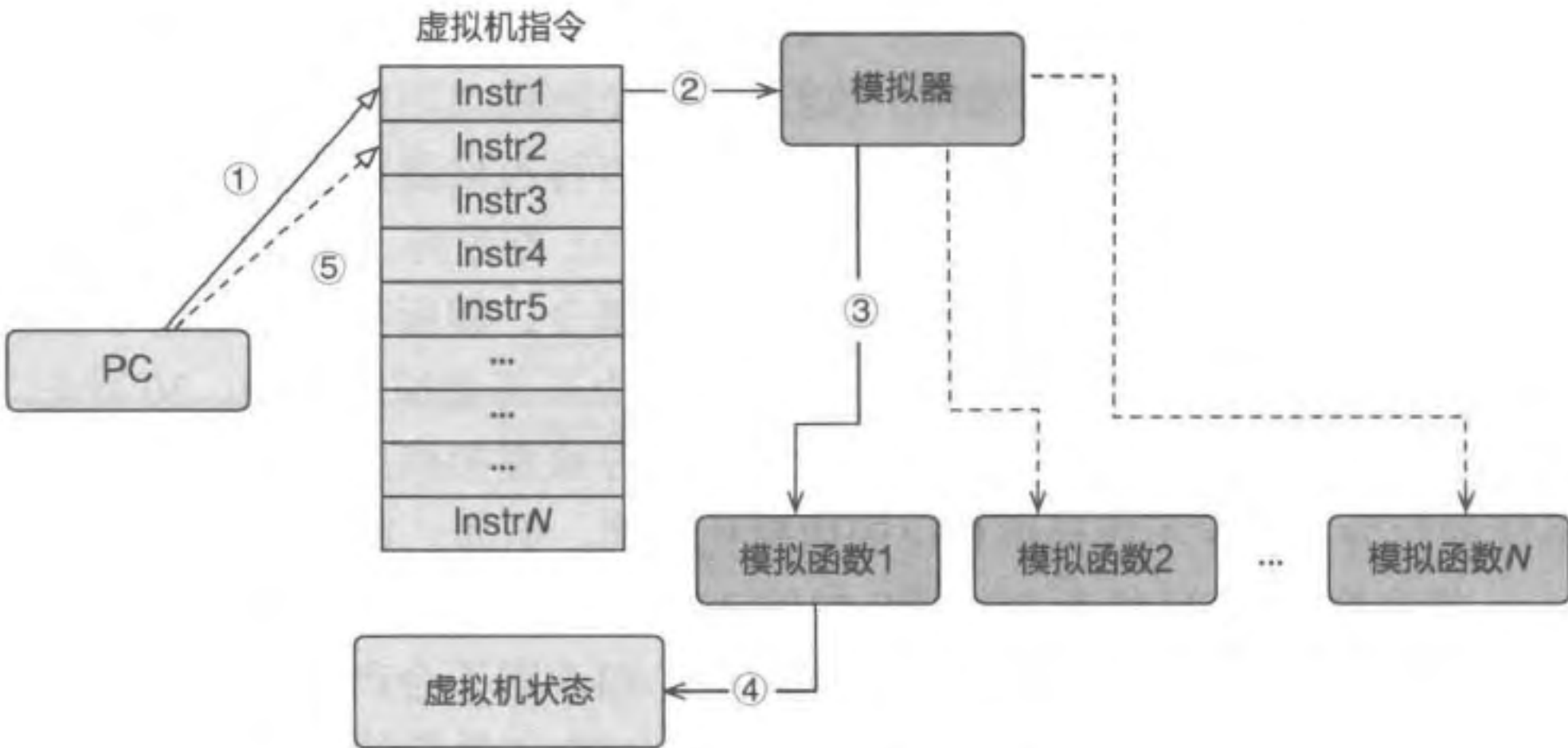


图 14.7 指令模拟的过程

例如，对于 AArch32 中使用 cps 关闭中断的操作，具体的模拟方法是修改模拟的 PSTATE。此后，虚拟机监控器将根据 PSTATE 中 I 或 F 位的情况决定中断的转发。如果两位均为 1，则停止将虚拟 IRQ 和 FIQ 中断插入虚拟机。

解释执行的方法不仅可以用于模拟与当前物理主机 ISA 相同的虚拟机，也可以模拟不同 ISA 的虚拟机。尽管这种方法可以解决不可虚拟化架构的硬件缺陷，但由于它不加区分地模拟每条指令，给虚拟机的执行带来了巨大的性能开销。

14.3.3 动态二进制翻译

解释执行方法的性能开销主要来自不加区分地依次模拟每一条指令。动态二进制翻译技术通过将多条指令直接翻译成对应的模拟函数，然后直接执行翻译后的代码，从而提高了性能。图 14.8 展示了动态二进制翻译的简要过程。动态二进制翻译以基本块 (Basic Block) 为粒度^①，将一个基本块内的所有指令都翻译成最终的目标代码，敏感指令会在翻译过程中被替换。

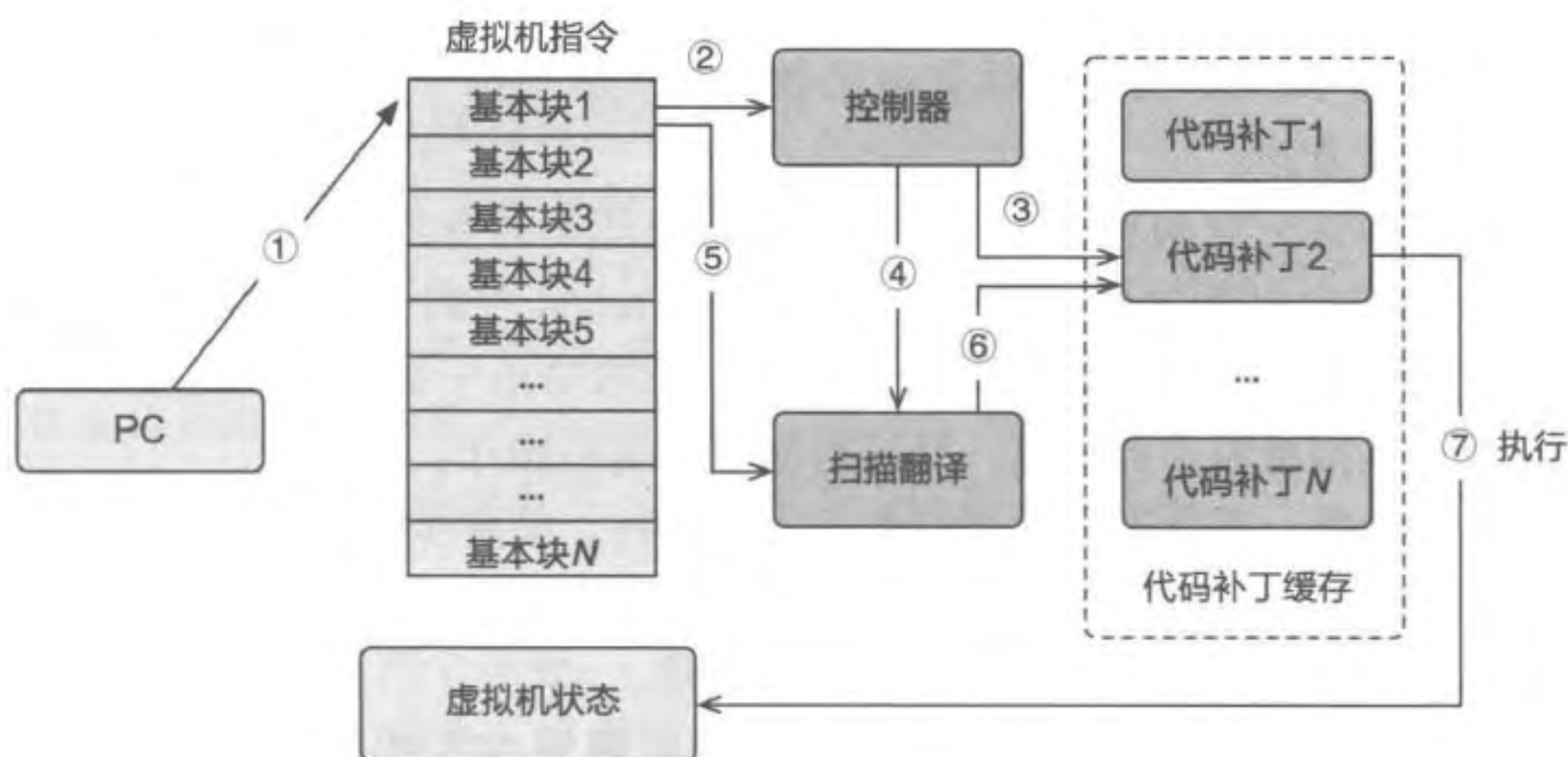


图 14.8 动态二进制翻译的过程

动态二进制翻译的简要过程如下：

- 第一步：虚拟机监控器读取虚拟机的 PC，得到 PC 所指向的基本块。
- 第二步：虚拟机监控器唤醒控制器。
- 第三步：控制器根据 PC 中的指令地址查找代码补丁缓存中是否存在已经翻译过的代码块，如果有，则直接跳到第七步。
- 第四步：如果缓存中不存在对应的代码块，则控制器唤醒扫描翻译模块。
- 第五步：扫描翻译模块读取内存中虚拟机的基本块，将其中的敏感指令替换成其

① 基本块是编译理论中的常见概念，它指一段按顺序执行的代码块，这段代码中除了最后一条指令外，中间没有任何改变控制流的指令（包括 bl、ret 等）。也就是说，基本块是只有一个入口和一个出口的代码块。

他指令。同时，还需要将此基本块中的最后一条指令也替换为一条跳转指令，以通知虚拟机监控器该基本块已经执行完（之后虚拟机监控器可取出下一个基本块并进行翻译和执行）。

- 第六步：翻译后的基本块称为代码补丁，将被放置于缓存中，以加速下次下陷过程。
- 第七步：直接执行代码补丁中的代码，并根据指令语义更新虚拟机状态（虚拟寄存器、内存、模拟设备）。

与解释执行的方法相比，动态二进制由于采用批量翻译以及缓存的思想，从而提高了 CPU 虚拟化的性能。实际使用时，可以采用一些方法来进一步加速二进制翻译的速度。例如，如果基本块执行之后的下一个基本块是确定的（如使用 b 等指令进行直接跳转），而且下一个基本块也被翻译过，则可以修改前一个基本块的最后一条指令为跳转指令，直接跳转到后一个基本块，这样便可绕过控制器。

14.3.4 扫描 - 翻译

实际上，如果宿主机的指令集与虚拟机相同，那么非敏感指令将不需要模拟，可直接在 CPU 中执行。因此，另一种优化方法就是只让敏感指令下陷，其他指令直接执行，这就是扫描 - 翻译方法。

那么，该如何区分这两种指令？在执行虚拟机代码时首先扫描代码块，将其中的敏感指令替换成一定会触发异常下陷的指令（如 AArch64 中的 svc 指令），其他指令保持不变。执行代码时，大部分指令可直接执行，敏感指令由于被替换为触发异常的特权指令，因此能够被虚拟机监控器捕捉，并进行对应的模拟操作。

在虚拟机执行前，其所有内存被设置为不可执行。简要步骤如下：

- 第一步：物理 CPU 中的 PC 第一次执行某个代码页中的指令时，由于该代码页被设置为不可执行，因此会触发一次缺页异常。
- 第二步：此异常改变 CPU 特权级为 EL1，调用虚拟机监控器的缺页异常处理函数。
- 第三步：异常处理函数将控制流交给控制器，它根据下陷地址找到触发此次异常的指令地址，控制器首先查找缓存中是否存在已经翻译过的代码页，如果有则直接返回用户态。
- 第四步：如果缓存中不存在代码页，则模拟器唤醒扫描翻译模块。
- 第五步：扫描翻译模块读取内存中待翻译的代码页内容，并将其中的敏感指令（如果存在）替换为触发下陷的特权指令。
- 第六步：翻译后的代码页被放置于缓存中。
- 第七步：虚拟机监控器将翻译完的代码页的权限设置为可执行，回到用户态。
- 第八步：虚拟机执行翻译后的代码，此时大部分指令都可以直接执行。如果遇到替换后的指令，则下陷通知虚拟机监控器并完成相应的指令模拟操作。

实际上，由于敏感指令只可能存在于操作系统内核的代码中，用户态通常不会包含这样的指令。因此，可只扫描内核代码，而忽略所有用户进程的代码，从而进一步提升性能。

14.3.5 半虚拟化技术

以上介绍的三种方法都假设不能修改客户虚拟机的源代码，因而必须在机器指令层面进行模拟或翻译。这种无须修改客户虚拟机源码的方式被称为全虚拟化（Full Virtualization）技术，而允许修改客户虚拟机源码的方式则被称为半虚拟化（Para-virtualization）技术。

半虚拟化技术需要客户操作系统与虚拟机监控器进行协同设计。一方面，虚拟机监控器为虚拟机提供超级调用（Hypercall），这些超级调用和系统调用类似，涵盖了调度、内存、I/O 等多方面的功能。另一方面，客户操作系统的代码需要修改，即替换不可虚拟化的指令，将其更改为超级调用。

小知识：剑桥大学的 Xen 虚拟机监控器

2003 年，剑桥大学基于半虚拟化技术的思想设计并实现了 Xen 虚拟机监控器。在传统的全虚拟化技术中，无须修改客户虚拟机，它甚至不知道自己运行在虚拟化环境内。而在半虚拟化技术中，客户虚拟机不仅知道自己运行在虚拟化环境内，还要对此进行相应的修改，从而绕开 x86 的缺陷。例如，将 17 条敏感指令直接替换成可以下陷的指令。此外，Xen 还设计了一种高效的 I/O 虚拟化机制。

半虚拟化技术有以下两个优势：

- 带来更高的性能。首先，半虚拟化无须模拟运行敏感指令。其次，半虚拟化可减少冗余的代码逻辑、数据拷贝、特权级切换等操作，例如，虚拟机可通过超级调用将调度信息传递至虚拟机监控器进行协同，以减少调度开销。
- 缓解语义鸿沟（Semantic Gap）问题。不使用半虚拟化技术的时候，虚拟机监控器只能看到内存中的二进制数据，难以将这些二进制数据转化成有意义的语义。半虚拟化允许虚拟机监控器获得虚拟机内部的状态，因此可以进一步提高资源的分配效率。

然而，由于半虚拟化技术需要修改客户操作系统源码，尤其是需要修改不同操作系统的不同版本，这会带来较大的开发和调试成本。而对于非开源的操作系统，也较难在其中添加半虚拟化的支持。

14.3.6 硬件虚拟化技术

前几节主要介绍不可虚拟化架构的软件解决方案，本节我们将介绍基于硬件的解

决方案。

Intel 和 AMD 于 2005 和 2006 年相继推出虚拟化的硬件扩展，解决不可虚拟化架构的缺陷。图 14.9 展示了 Intel VT-x 的硬件虚拟化架构图。硬件虚拟化技术在已有 CPU 特权级下新增了两个模式，分别是根模式（Root Mode）和非根模式（Non-root Mode）。作为最高特权的管理软件，虚拟机监控器管理所有物理资源，并使用硬件虚拟化的功能为上层虚拟机提供服务。虚拟机监控器和虚拟机分别运行在根模式和非根模式。为了管理虚拟机的硬件行为，Intel VT-x 为每一个虚拟机提供虚拟机控制结构（Virtual Machine Control Structure, VMCS）。虚拟机监控器通过配置虚拟机控制结构来管理虚拟机的内存映射和其他行为。通过这种模式，过去不会引起下陷的敏感指令都能被运行在根模式的虚拟机监控器捕捉，继而实现相应的虚拟化操作。

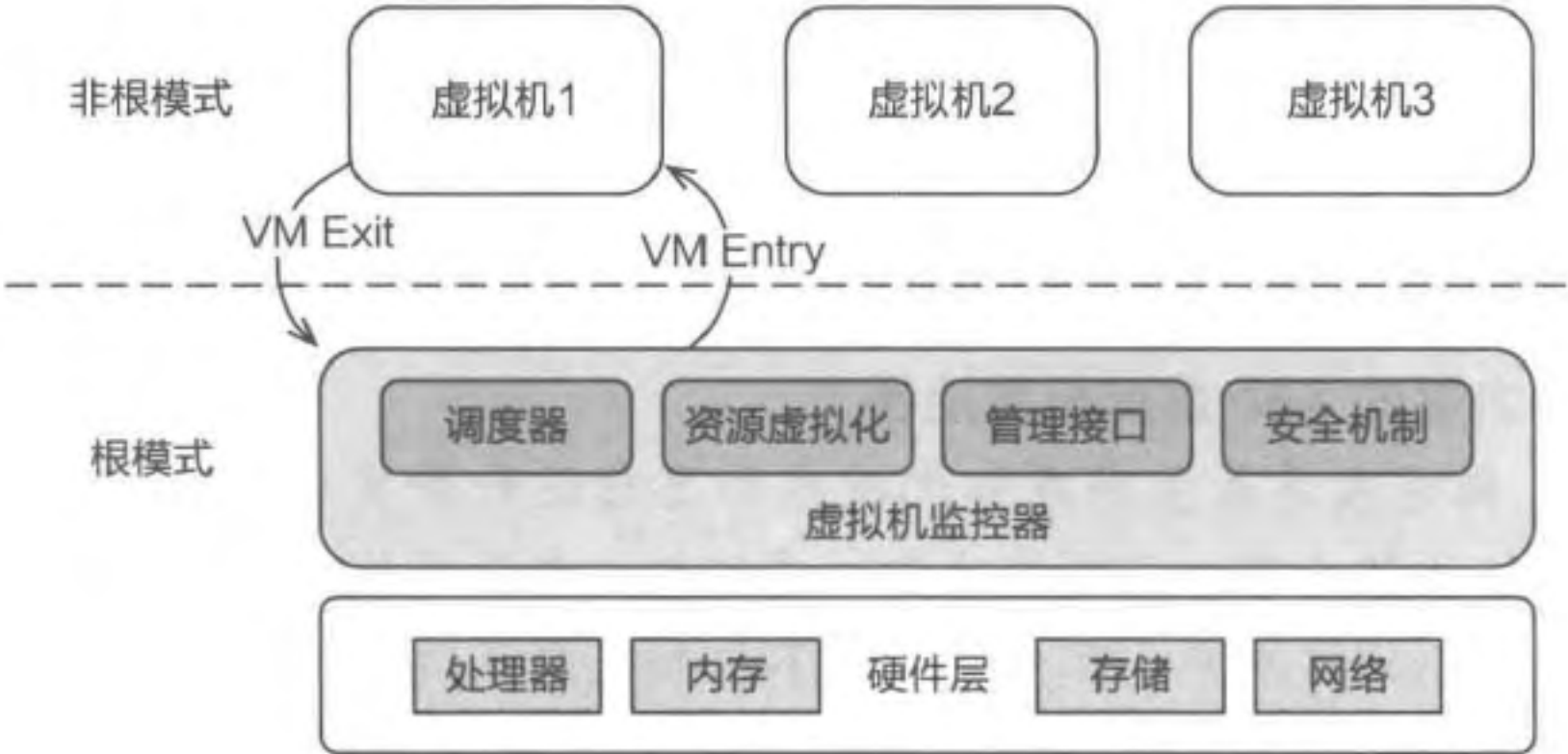


图 14.9 Intel VT-x 硬件虚拟化架构

类似地，ARM 公司于 2012 年推出了硬件虚拟化扩展。图 14.10 展示了 ARMv8.0 硬件虚拟化扩展的架构图，其中引入了一个全新的特权级——EL2，它是除了 EL3 之外最高的特权级。在此特权级中运行的是虚拟机监控器，它控制所有的物理资源，并管理运行在非特权级（也就是运行在 EL0 和 EL1）中的软件。

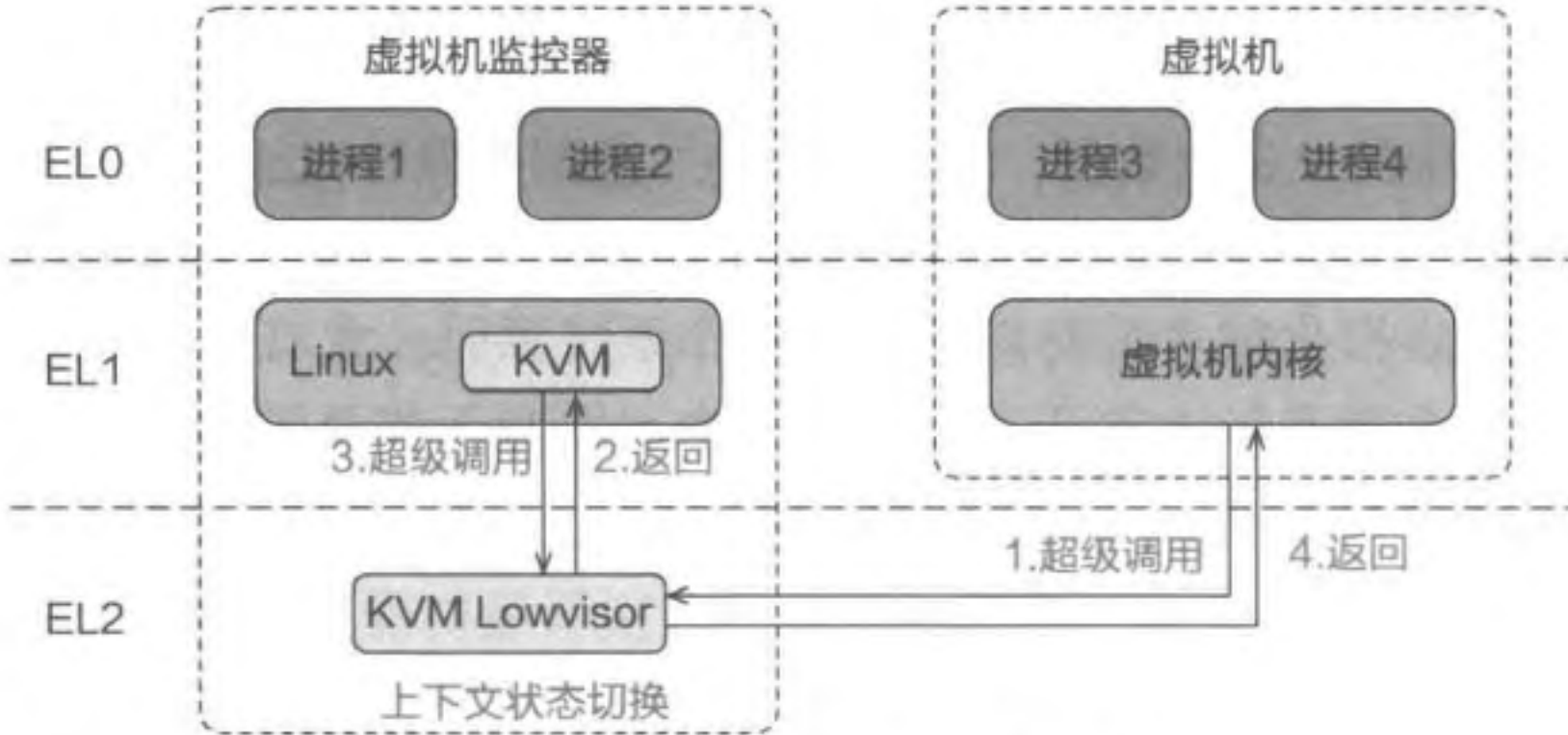


图 14.10 ARMv8.0 硬件虚拟化扩展

硬件虚拟化对敏感指令的支持主要体现在两个方面。首先，一部分敏感指令不再需要下陷，而是被直接重定向到虚拟硬件部件。例如，在 AArch32 中，客户操作系统直接运行在 EL1 内，因而 cps 指令能够直接修改 PSTATE 寄存器，再也不会被硬件忽略。其次，对于过去不可虚拟化架构中其他的敏感指令，在引入硬件虚拟化技术之后，这些指令都可以通过配置的方式引起下陷，从而能够被虚拟机监控器处理。在 ARM 硬件虚拟化中，硬件为虚拟机监控器提供了一个名为 HCR_EL2 的系统寄存器。此寄存器中的不同比特位决定了虚拟机的不同行为。例如，其中第 13 位决定虚拟机执行 wfi 指令是否会引起虚拟机下陷^①。如果虚拟机监控器能够捕捉到这条指令，可将 CPU 的控制权收回，继而调度其他虚拟机。

在引入硬件虚拟化技术后，虚拟机可直接使用 CPU 中的寄存器，而无须使用软件模拟的寄存器。在 ARM 硬件虚拟化中，硬件为 EL1 以及 EL2 提供两套系统寄存器。这也意味着，如果发生虚拟机下陷，虚拟机监控器无须保存虚拟机在 EL1 中使用的系统寄存器。此外，运行在 EL2 的虚拟机监控器可以读写 EL1 和 EL0 的任何寄存器。因而在处理虚拟机下陷时，它可以随时查看和改变虚拟机的寄存器状态。请注意，在发生 vCPU 上下文切换时，虚拟机监控器依然需要将该 vCPU 在 EL0 和 EL1 中的所有系统寄存器以及通用寄存器都保存在内存中，并将即将运行的 vCPU 的相关寄存器加载至物理寄存器中。

小知识：x86 硬件虚拟化技术中的虚拟机下陷

每当虚拟机执行敏感指令时，虚拟机下陷触发，CPU 模式随之由非根模式切换为根模式。与 ARM 硬件虚拟化不同的是，x86 CPU 中的不同特权级共享同一份系统寄存器。因此，虚拟机下陷触发时，硬件会将 vCPU 的所有系统寄存器（例如 CR0、CR3 等）保存到 VMCS 中。

AArch64 硬件虚拟化技术的发展经过了两个阶段。第一个阶段是 ARMv8.0 版本。此版本中引入了全新的 EL2 特权级，该特权级中只能运行 Type-1 虚拟机监控器，不支持 Type-2 虚拟机监控器，如 KVM。正如前文所介绍的，Type-2 虚拟机监控器高度依赖宿主操作系统的功能。KVM 与 Linux 耦合在一起，两者运行在同一特权级和内存空间中。然而，Linux 在开发时被假设运行在 EL1，因而它只使用 EL1 下的系统寄存器（例如 TTBR0_EL1 和 TTBR1_EL1），但这些系统寄存器在 EL2 下并不存在^②。因此，在 ARMv8.0 的硬件虚拟化中，必须将 KVM 的一部分功能从 Linux 中剥离出来，以 Lowvisor 的形式运行在 EL2 中。宿主操作系统 Linux 和其他 KVM 的功能则依然运行在

① wfi 指令使得硬件进入低功耗状态，执行此指令通常表明操作系统已“无事可做”。

② EL2 中对应的页表基地址寄存器为 TTBR0_EL2 和 TTBR1_EL2。

EL1 中，如图 14.10 所示。

当发生虚拟机下陷时（例如虚拟机执行超级调用），首先下陷到 EL2，这是由硬件虚拟化的机制决定的。而 EL2 中的 KVM Lowvisor 在处理此次虚拟机下陷的过程中可能需要使用 Linux 的功能，所以它将控制流转回到 EL1 的 KVM 里，并调用 Linux 的功能。在此之后，KVM 重新回到 Lowvisor，并最终回到虚拟机内。可以看到，这样的架构设计造成 KVM 和 Lowvisor 之间的多次特权级切换，因而对虚拟机的运行性能带来了一定开销。

为了解决上述性能问题，第二阶段的 ARMv8.1 中进一步扩展了硬件虚拟化的功能，并增加了 VHE（Virtualization Host Extension）。VHE 意味着可在 EL2 中运行完整的宿主操作系统。此时 KVM 的架构也发生了相应转变，如图 14.11 所示。因此，在 ARMv8.1 之后，KVM 和宿主操作系统 Linux 的交互都不会带来任何特权级切换，从而优化了虚拟化的性能。

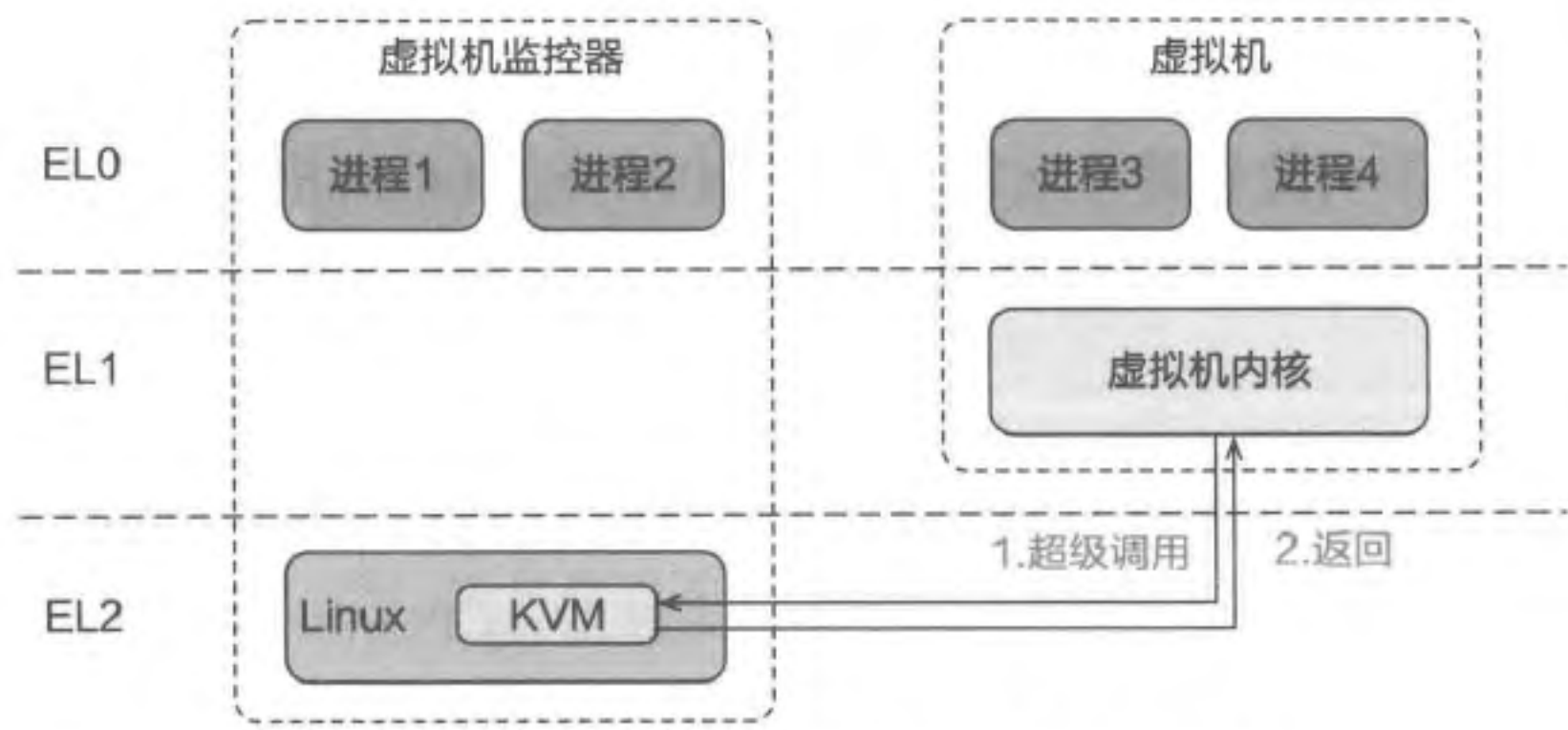


图 14.11 ARMv8.1 VHE 硬件虚拟化扩展

除了上述功能外，硬件虚拟化还对内存虚拟化提供了相关支持。我们将在下一节详细介绍。

14.3.7 小结

本节介绍了实现 CPU 虚拟化的 5 种技术，这些技术可以分为两大类，如表 14.3 所示。在硬件虚拟化技术出现之前，CPU 虚拟化技术主要通过软件技术解决不可虚拟化架构带来的问题，而在硬件虚拟化技术出现之后，CPU 虚拟化技术主要依托于硬件提供的虚拟化功能，例如通过配置 HCR_EL2 寄存器控制虚拟机的下陷行为等。

表 14.3 CPU 虚拟化技术总结

软件技术	硬件虚拟化技术
解释执行	新的硬件特权级，所有敏感指令可下陷
动态二进制翻译	
扫描 - 翻译	
半虚拟化技术	

上述技术在性能以及是否需要修改虚拟机代码等方面都存在着各自的优缺点，因此

有着相应的适用范围。此外，在硬件虚拟化技术推出之后，曾经的软件技术也未过时，依然存在适用场景。首先，当物理主机 ISA 和虚拟机 ISA 不同时，将不能使用硬件虚拟化技术，只能选择软件技术，例如在 x86 的机器上模拟 ARM 虚拟机。其次，在使用硬件虚拟化技术之后，依然可以组合使用某些软件技术，例如可以使用半虚拟化技术优化 I/O 场景下的性能，我们将在 14.5 节进行介绍。

14.4 内存虚拟化

本节主要知识点

- 为什么需要内存虚拟化？
- 内存虚拟化中的三种内存地址分别是什么？
- 如何通过影子页表和直接页表映射机制实现内存虚拟化？
- AArch64 架构中的两阶段地址翻译机制如何工作？
- 基于内存虚拟化的换页机制如何实现？

为什么需要内存虚拟化？我们首先从没有系统虚拟化的场景说起。使用系统虚拟化之前，操作系统内核直接运行在最高特权级，因此它有权限管理整个物理地址空间，“看到的”的是从零地址开始连续增长的物理地址空间。使用系统虚拟化之后，一台物理主机上可同时运行多台虚拟机，此时每个客户操作系统“看到的”物理地址空间应该从零开始连续增长（否则虚拟机无法正常运行）。此外，虚拟机监控器不允许客户操作系统访问不属于它的物理内存区域。如果此时某个客户操作系统可以访问任意物理内存区域，它就能读取甚至修改其他虚拟机以及虚拟机监控器内存中的数据，这破坏了虚拟机的内存隔离，严重威胁整个系统的安全。

因此，内存虚拟化需要满足两个需求：

- 第一，为每台虚拟机提供从零地址开始连续增长的物理地址空间。
- 第二，实现虚拟机之间的内存隔离，每台虚拟机只能访问分配给它的物理内存区域。

该如何满足这两个需求？这里介绍一种全新类型的内存地址——客户物理地址，这是虚拟机内使用的物理地址，不是真实的在总线中访存的地址。真正访存的物理地址改称为主机物理地址。在虚拟机的执行过程中，需要将客户物理地址转换成主机物理地址，然后通过后者访问存储在内存中的数据和代码。相应地，虚拟机监控器需要提供一种新的地址翻译机制，将客户物理地址翻译成主机物理地址。如图 14.12 所示，到目前为止，我们接触了三种不同类型的地址：

- 第一种是进程和客户操作系统使用的客户虚拟地址（Guest Virtual Address，